CS 0007: Introduction to Java Lecture 5

Nathan Ong
University of Pittsburgh
September 15, 2016



and the Application Programming Interface (API)

STRINGS

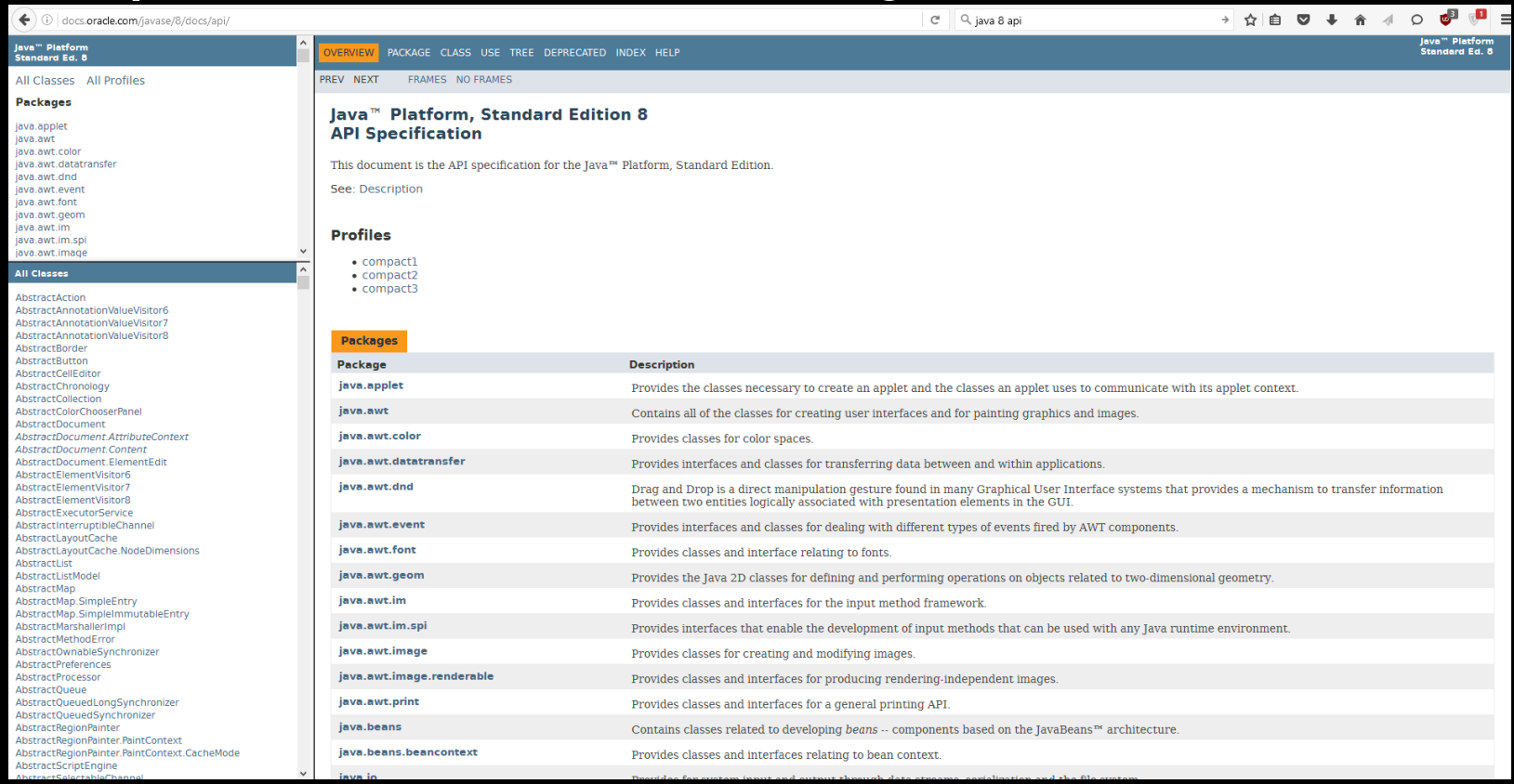
Strings

- A length of characters encased in quotation marks.

```
String name = "Nathan";
```

Application Programming Interface

- The Java “dictionary”
- http://docs.oracle.com/javase/8/docs/



What Can I Do with It?

- Look up relevant classes (e.g. String) for their relevant functions/methods.

An Example

- What is the 4th character of `"Nathan"` and how do I get it?
- Check the API

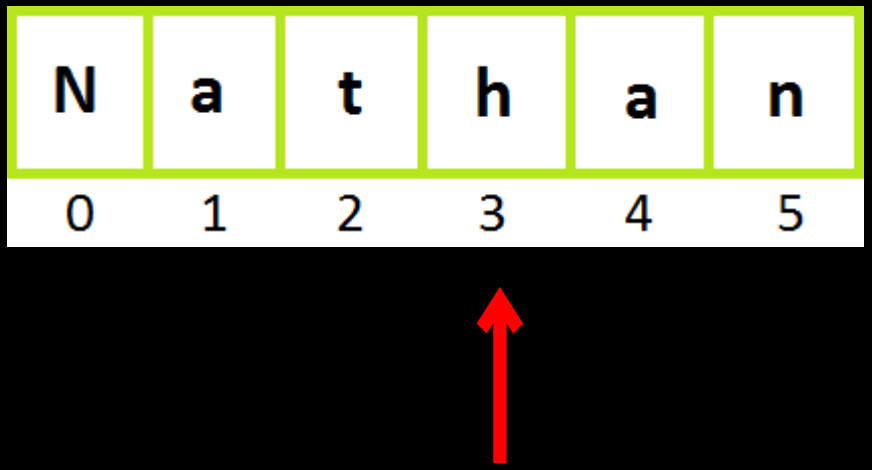
Code Example

```
public class Name
{
	public static void main(String[] args)
	{
		String name = "Nathan";
		System.out.println(name.charAt(4));
	}//end method main
}//end class Name
```

There’s one slight issue

Counting in Java

The first character in the series is at the 0th index

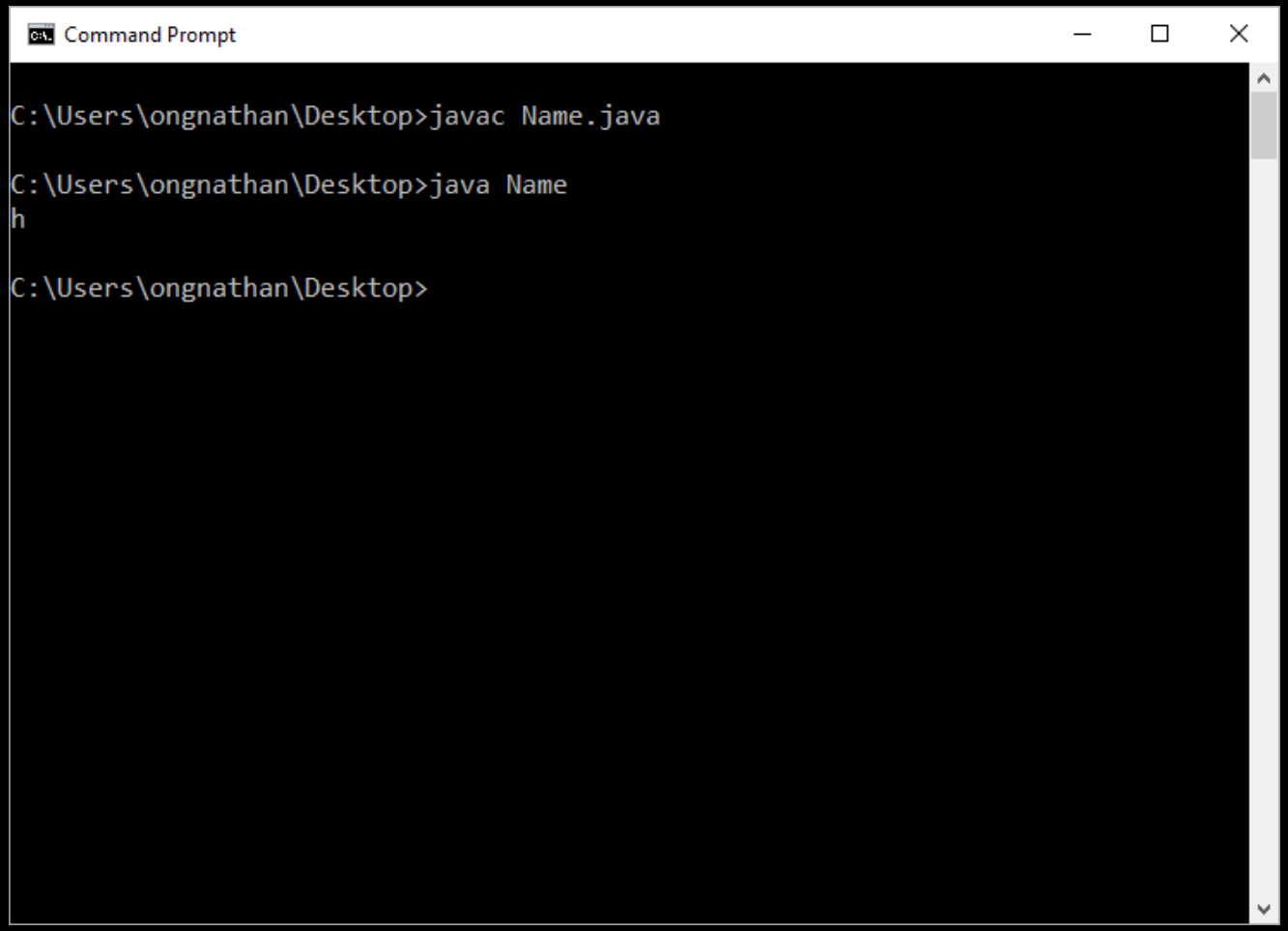


Code Example

```
public class Name
{
	public static void main(String[] args)
	{
		String name = "Nathan";
		System.out.println(name.charAt(3));
	}//end method main
}//end class Name
```

The 4th character is at the 3rd index

Code Example

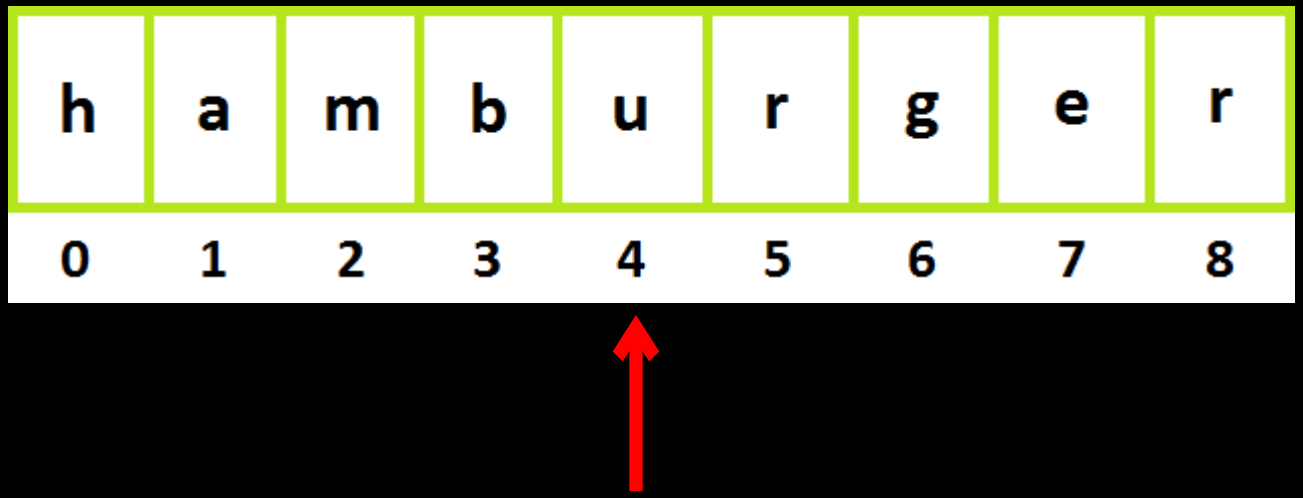


Another Example

- How do I get "urge" from "hamburger"?
- (It’s a substring)
- Check the API

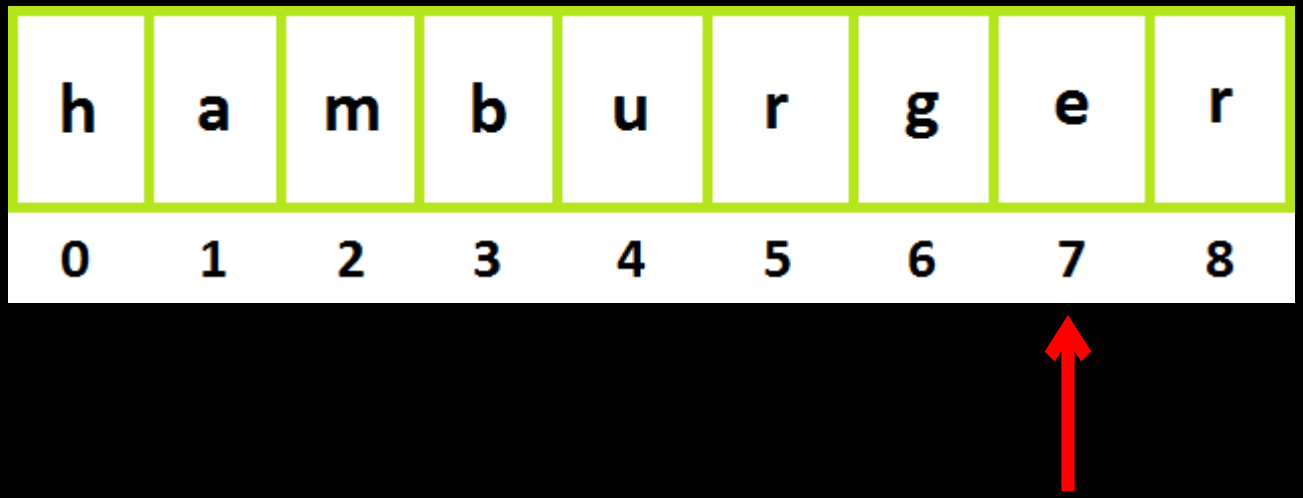
Re-Counting in Java

The first character in the substring is at the 4th index



Re-Counting in Java

The last character in the substring is at the 7th index



Another Example

```
public class DriveThru
{
  public static void main(String[] args)
  {
  String fastFood = "hamburger";
  System.out.println(fastFood.substring(4,7));
  }//end method main
}//end class DriveThru
```

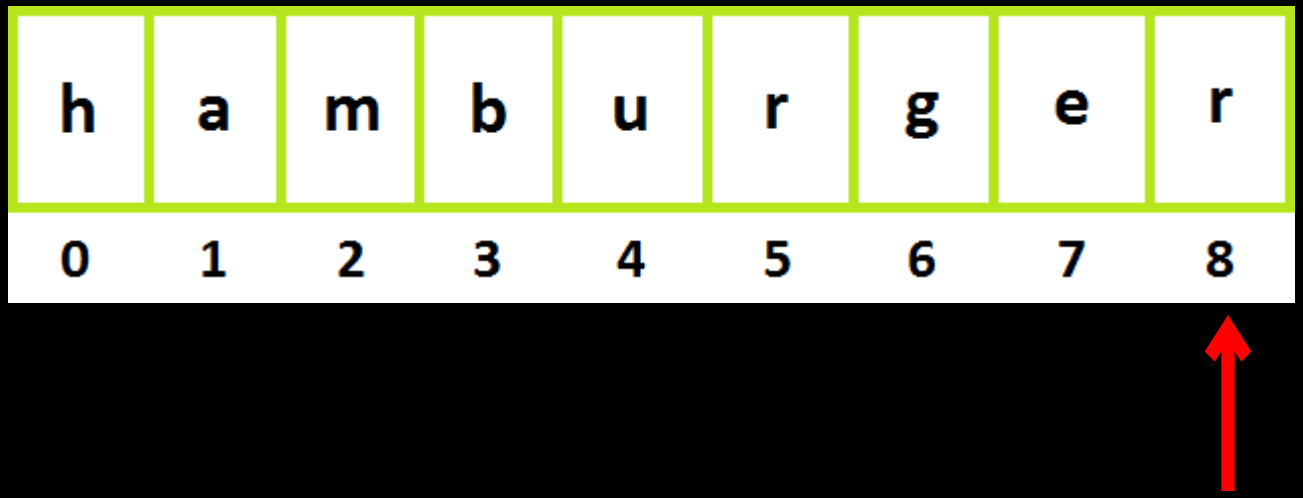
There’s one slight issue

“The substring begins at the specified beginIndex and extends to the character at index endIndex – 1”

Where can I find this information?

Re-Counting in Java

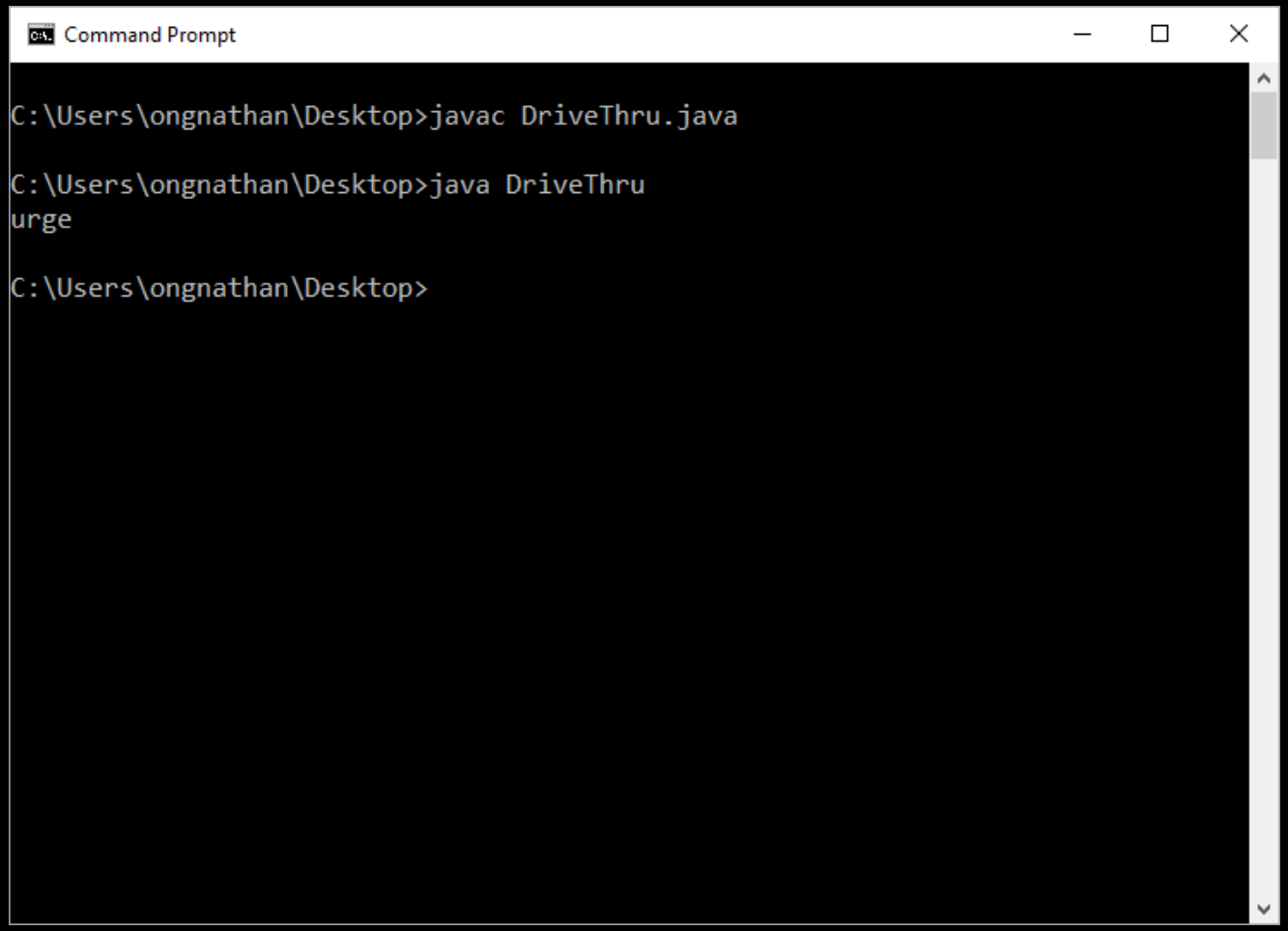
The last character in the substring is at the 7th index endIndex – 1 = 7



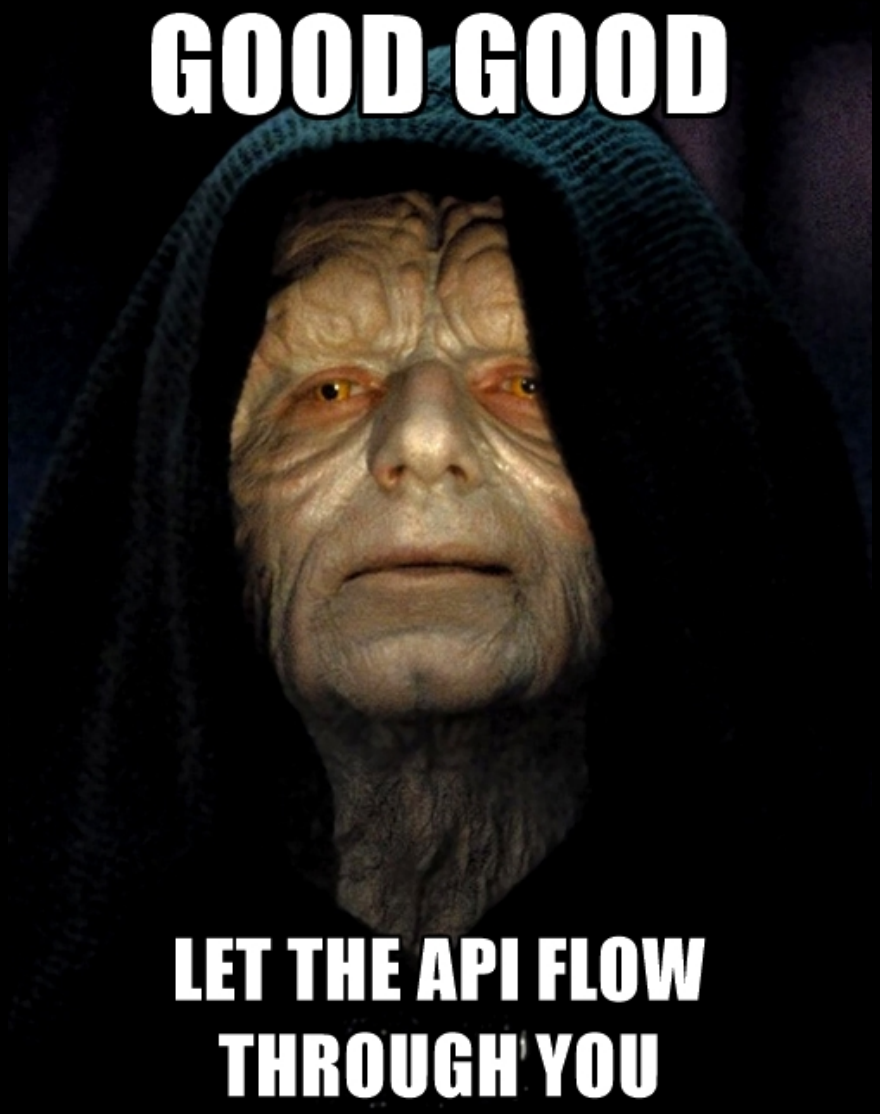
Another Example

```
public class DriveThru
{
  public static void main(String[] args)
  {
  String fastFood = "hamburger";
  System.out.println(fastFood.substring(4,8));
  }//end method main
}//end class DriveThru
```

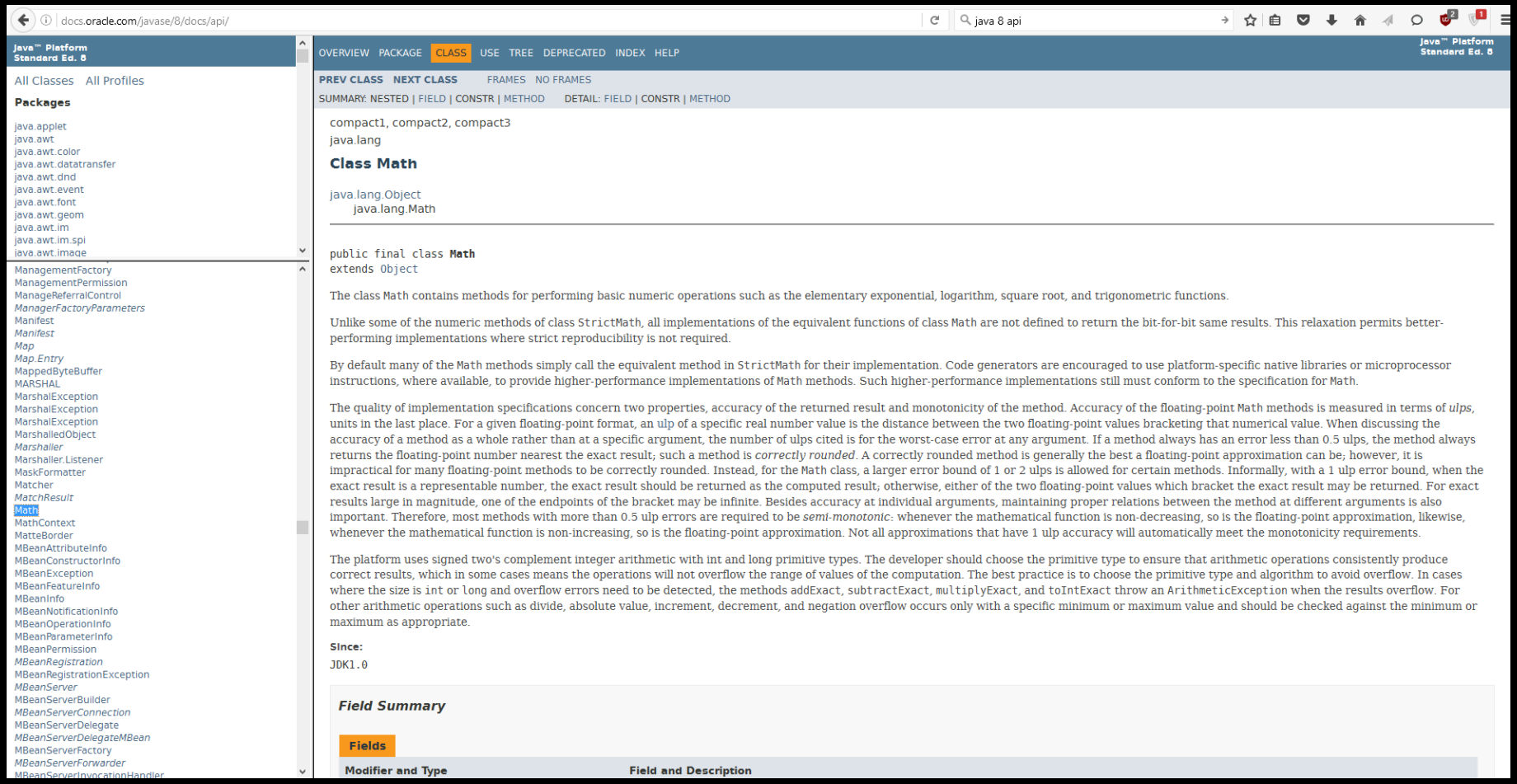
Another Example



You must learn to embrace the API



Math Class


docs.oracle.com/javase/8/docs/api/
java 8 api
Java™ Platform Standard Ed. 8
All Classes All Profiles
Packages
java.applet
java.awt
java.awt.color
java.awt.datatransfer
java.awt.dnd
java.awt.event
java.awt.font
java.awt.geom
java.awt.im
java.awt.im.spi
java.awt.image
ManagementFactory
ManagementPermission
ManageReferralControl
ManagerFactoryParameters
Manifest
Manifest
Map
Map.Entry
MappedByteBuffer
MARSHAL
MarshalException
MarshalException
MarshalException
MarshalledObject
Marshaller
Marshaller.Listener
MaskFormatter
Matcher
MatchResult
Math
MathContext
MatteBorder
MBeanAttributeInfo
MBeanConstructorInfo
MBeanException
MBeanFeatureInfo
MBeanInfo
MBeanNotificationInfo
MBeanOperationInfo
MBeanParameterInfo
MBeanPermission
MBeanRegistration
MBeanRegistrationException
MBeanServer
MBeanServerBuilder
MBeanServerConnection
MBeanServerDelegate
MBeanServerDelegateMBean
MBeanServerFactory
MBeanServerForwarder
MBeanServerInvocationHandler
OVERVIEW PACKAGE CLASS USE TREE DEPRECATED INDEX HELP
Java™ Platform Standard Ed. 8
PREV CLASS NEXT CLASS FRAMES NO FRAMES
SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD
compact1, compact2, compact3
java.lang
Class Math
java.lang.Object
java.lang.Math
public final class Math
extends Object
The class Math contains methods for performing basic numeric operations such as the elementary exponential, logarithm, square root, and trigonometric functions.
Unlike some of the numeric methods of class StrictMath, all implementations of the equivalent functions of class Math are not defined to return the bit-for-bit same results. This relaxation permits better-performing implementations where strict reproducibility is not required.
By default many of the Math methods simply call the equivalent method in StrictMath for their implementation. Code generators are encouraged to use platform-specific native libraries or microprocessor instructions, where available, to provide higher-performance implementations of Math methods. Such higher-performance implementations still must conform to the specification for Math.
The quality of implementation specifications concern two properties, accuracy of the returned result and monotonicity of the method. Accuracy of the floating-point Math methods is measured in terms of ulps, units in the last place. For a given floating-point format, an ulp of a specific real number value is the distance between the two floating-point values bracketing that numerical value. When discussing the accuracy of a method as a whole rather than at a specific argument, the number of ulps cited is for the worst-case error at any argument. If a method always has an error less than 0.5 ulps, the method always returns the floating-point number nearest the exact result; such a method is correctly rounded. A correctly rounded method is generally the best a floating-point approximation can be; however, it is impractical for many floating-point methods to be correctly rounded. Instead, for the Math class, a larger error bound of 1 or 2 ulps is allowed for certain methods. Informally, with a 1 ulp error bound, when the exact result is a representable number, the exact result should be returned as the computed result; otherwise, either of the two floating-point values which bracket the exact result may be returned. For exact results large in magnitude, one of the endpoints of the bracket may be infinite. Besides accuracy at individual arguments, maintaining proper relations between the method at different arguments is also important. Therefore, most methods with more than 0.5 ulp errors are required to be semi-monotonic: whenever the mathematical function is non-decreasing, so is the floating-point approximation, likewise, whenever the mathematical function is non-increasing, so is the floating-point approximation. Not all approximations that have 1 ulp accuracy will automatically meet the monotonicity requirements.
The platform uses signed two's complement integer arithmetic with int and long primitive types. The developer should choose the primitive type to ensure that arithmetic operations consistently produce correct results, which in some cases means the operations will not overflow the range of values of the computation. The best practice is to choose the primitive type and algorithm to avoid overflow. In cases where the size is int or long and overflow errors need to be detected, the methods addExact, subtractExact, multiplyExact, and toIntExact throw an ArithmeticException when the results overflow. For other arithmetic operations such as divide, absolute value, increment, decrement, and negation overflow occurs only with a specific minimum or maximum value and should be checked against the minimum or maximum as appropriate.
Since:
JDK1.0
Field Summary
Fields
Modifier and Type
Field and Description

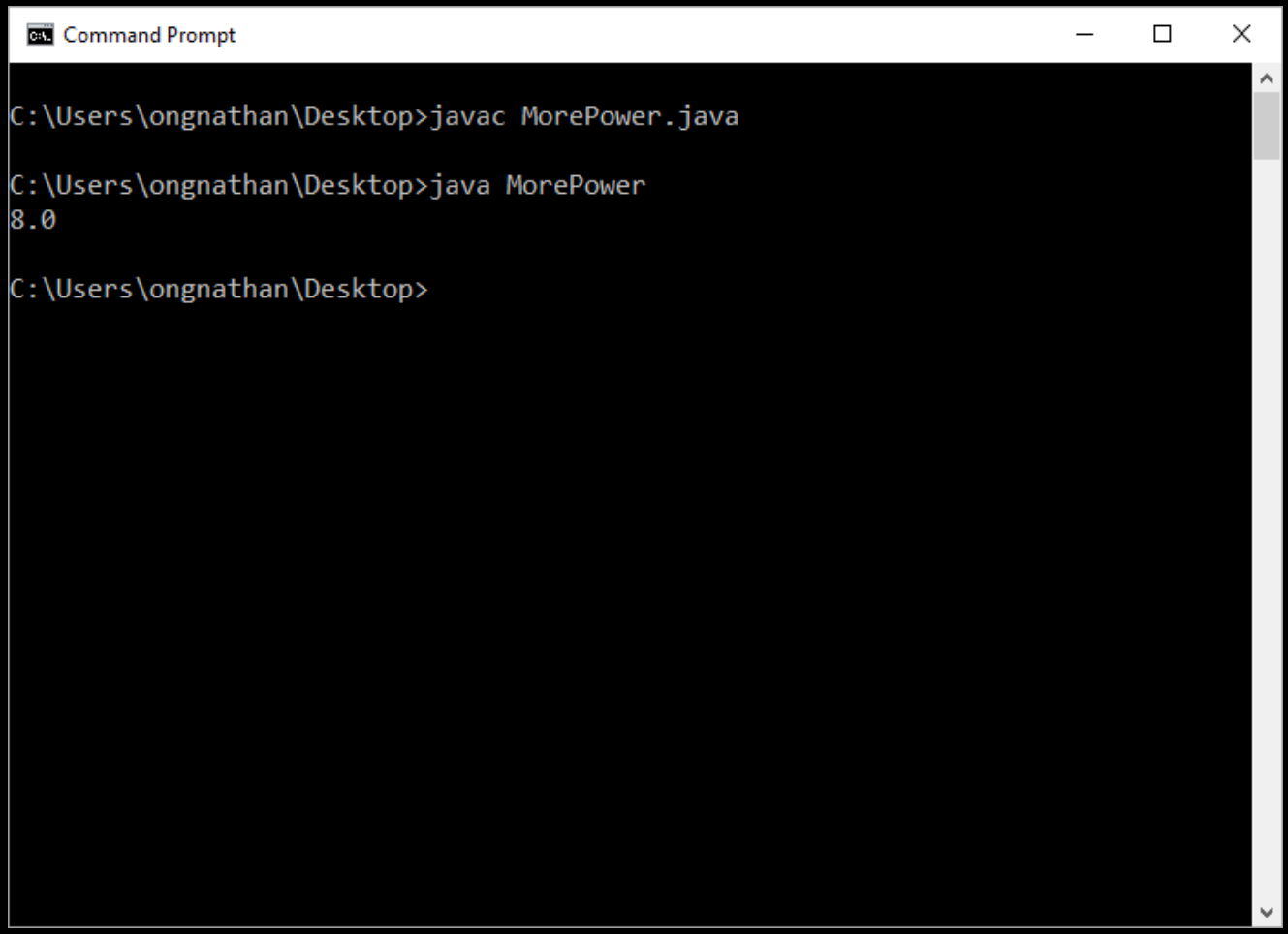
Remember from before

- The caret (^) does NOT indicate exponentiation
- In other words, in Java, 2^3 does not equal 8
- There is another way to do exponentiation; maybe it is in the Math class?

Math Example

```
public class MorePower
{
  public static void main(String[] args)
  {
  double base = 2.0;
  double exponent = 3.0;
  System.out.println(Math.pow(base,exponent));
  }//end method main
}//end class MorePower
```

Math Example



When to Use the Variable/Class Name

- Does the “Modifier and Type” column of the method say “`static`”?
  - Yes: Use Class name (typically uppercase first letter)
  - No: Use variable name (typically lowercase first letter)